Project Scope - Chatbot Widget

Table of Contents

Executive Summary

The **Chatbot Widget** is a production-ready, embeddable AI chatbot that can be integrated into any existing website. It provides users with a conversational AI assistant that can answer general questions, analyze uploaded documents, and generate files. The widget operates in three states: floating sphere, compact view, and full-screen expanded view.

Project Type: Full-stack web application

Duration: 8-12 weeks (estimated)

Team Size: 2-3 developers

Target Platform: Web (desktop, tablet, mobile)

Primary Technology: FastAPI (backend), React (frontend), PostgreSQL (database)

Project Overview

Vision

Create a seamless, professional chatbot widget that enhances user engagement on any website by providing intelligent conversational AI capabilities with document analysis and file generation features.

Objectives

1. **Build a reusable widget** that integrates into any website with minimal setup
2. **Provide intelligent chat** powered by Google Gemini 2.5 Flash LLM
3. **Enable document analysis** through RAG (Retrieval-Augmented Generation)
4. **Allow file generation** for summaries, reports, and exports
5. **Maintain conversation history** with a dashboard for users
6. **Ensure security** with authentication, encryption, and rate limiting
7. **Optimize performance** for fast response times and scalability

Key Features

- **Floating Widget** - Non-intrusive sphere at website footer
- **Compact View** - Minimized chat interface (350×300px)
- **Full-Screen View** - Expanded interface with dashboard
- **File Upload** - Support for PDF, TXT, DOCX, XLSX, Markdown
- **Chat Interface** - Real-time messaging with typing indicators
- **Message Editing** - Edit messages before sending
- **File Generation** - Generate summaries, reports, exports
- **Conversation Dashboard** - View, search, filter past conversations
- **Multi-LLM Support** - Gemini, Claude, GPT-4, DeepSeek
- **Authentication** - JWT-based user authentication
- **Rate Limiting** - Prevent abuse and control costs

In Scope

Core Features

1. Frontend (React)

Widget States:

- ✅ Floating sphere (60×60px) at website footer
- ✅ Compact widget (350×300px) with limited features
- ✅ Expanded full-screen view with 3-column layout

- ✅ Smooth transitions between states

Chat Interface:

- ✅ Message input with send button
- ✅ Message history display with timestamps
- ✅ Typing indicators
- ✅ User and assistant message differentiation
- ✅ Markdown rendering for responses

Message Editing:

- ✅ Edit button on user messages
- ✅ Edit mode with save/cancel options
- ✅ Show edited indicator
- ✅ Preserve edit history

File Upload:

- ✅ Drag-and-drop file upload
- ✅ File type validation
- ✅ File size validation (max 50MB)
- ✅ Upload progress indicator
- ✅ File preview

Dashboard:

- ✅ List of past conversations
- ✅ Search conversations by title/content
- ✅ Filter by date range
- ✅ Filter by file type
- ✅ Conversation preview
- ✅ Pin/unpin conversations
- ✅ Delete conversations
- ✅ Export conversation

Settings:

- ✅ Model provider selection
- ✅ Temperature adjustment

- ✅ Max tokens configuration
- ✅ Theme selection (light/dark)

Responsive Design:

- ✅ Desktop layout (1920px+)
- ✅ Tablet layout (768px-1024px)
- ✅ Mobile layout (320px-767px)
- ✅ Collapsible sidebars on mobile

2. Backend (FastAPI)

Authentication:

- ✅ User registration
- ✅ User login
- ✅ JWT token generation
- ✅ Token refresh
- ✅ Password hashing with bcrypt
- ✅ Role-based access control (user, admin)

Chat Endpoints:

- ✅ Send message
- ✅ Get conversation history
- ✅ Edit message
- ✅ Delete message
- ✅ Create conversation
- ✅ Archive conversation

File Endpoints:

- ✅ Upload file
- ✅ List files
- ✅ Get file preview
- ✅ Delete file
- ✅ Download file

LLM Integration:

- ✅ Google Gemini 2.5 Flash integration

- ✅ Multi-model support (OpenAI, Anthropic, DeepSeek)
- ✅ Streaming responses
- ✅ Token counting
- ✅ Error handling and retries

RAG Pipeline:

- ✅ Document parsing (PDF, TXT, DOCX, XLSX, Markdown)
- ✅ Text chunking
- ✅ Embedding generation
- ✅ Vector store (FAISS)
- ✅ Semantic search
- ✅ Context retrieval

File Generation:

- ✅ Generate summary
- ✅ Generate report
- ✅ Export conversation as PDF
- ✅ Export conversation as TXT
- ✅ Generate file from chat context

Caching:

- ✅ Redis caching for embeddings
- ✅ Response caching
- ✅ User session caching

Rate Limiting:

- ✅ API rate limiting (requests per minute)
- ✅ File upload rate limiting
- ✅ LLM API call limiting
- ✅ Per-user quotas

3. Database (PostgreSQL)

Tables:

- ✅ users
- ✅ conversations

- ✅ messages
- ✅ uploaded_files
- ✅ file_chunks
- ✅ conversation_files
- ✅ audit_logs

Features:

- ✅ Proper indexing for performance
- ✅ Foreign key constraints
- ✅ Data validation
- ✅ Timestamps (created_at, updated_at)
- ✅ Soft deletes (is_archived)

4. Security

Authentication & Authorization:

- ✅ JWT-based authentication
- ✅ Role-based access control
- ✅ Secure password hashing
- ✅ Token expiration
- ✅ Refresh token mechanism

Data Protection:

- ✅ HTTPS/TLS encryption
- ✅ Environment variable management
- ✅ API key encryption
- ✅ Sensitive data masking in logs

API Security:

- ✅ CORS configuration
- ✅ CSRF protection
- ✅ Rate limiting
- ✅ Input validation
- ✅ SQL injection prevention

- ✅ XSS protection

Infrastructure Security:

- ✅ Firewall rules
- ✅ Network isolation
- ✅ Secrets management
- ✅ Regular security audits

5. Performance & Scalability

Performance:

- ✅ Response time < 500ms for chat
- ✅ File upload handling (up to 50MB)
- ✅ Efficient database queries
- ✅ Caching strategy
- ✅ CDN for static assets

Scalability:

- ✅ Horizontal scaling (multiple backend instances)
- ✅ Load balancing
- ✅ Database connection pooling
- ✅ Async processing for long-running tasks
- ✅ Message queue for background jobs

6. Monitoring & Logging

Logging:

- ✅ Application logs
- ✅ API request/response logs
- ✅ Error logs with stack traces
- ✅ Audit logs for security events
- ✅ Performance metrics

Monitoring:

- ✅ Health check endpoints
- ✅ Uptime monitoring

- ✅ Error rate tracking
- ✅ Response time metrics
- ✅ Resource utilization (CPU, memory)

Alerting:

- ✅ High error rate alerts
- ✅ Downtime alerts
- ✅ Performance degradation alerts
- ✅ Security incident alerts

7. Testing

Unit Tests:

- ✅ Backend service tests
- ✅ Authentication tests
- ✅ File processing tests
- ✅ LLM integration tests

Integration Tests:

- ✅ API endpoint tests
- ✅ Database tests
- ✅ LLM API tests

End-to-End Tests:

- ✅ User workflows
- ✅ File upload and processing
- ✅ Chat conversations
- ✅ Dashboard functionality

Performance Tests:

- ✅ Load testing
- ✅ Stress testing
- ✅ Scalability testing

8. Documentation

Developer Documentation:

- ✅ API documentation (Swagger/OpenAPI)
- ✅ Backend setup guide
- ✅ Frontend setup guide
- ✅ Database schema documentation
- ✅ Architecture diagrams

User Documentation:

- ✅ Widget integration guide
- ✅ User manual
- ✅ FAQ
- ✅ Troubleshooting guide

Operational Documentation:

- ✅ Deployment guide
- ✅ Monitoring guide
- ✅ Backup & recovery guide
- ✅ Scaling guide

9. Deployment

Development Environment:

- ✅ Local development setup
- ✅ Docker Compose for local testing
- ✅ Hot reload for development

Staging Environment:

- ✅ Pre-production testing
- ✅ Performance testing
- ✅ Security testing

Production Environment:

- ✅ Cloud deployment (AWS/GCP/Azure)
- ✅ CI/CD pipeline
- ✅ Automated backups
- ✅ Disaster recovery

Out of Scope

Features NOT Included

1. Advanced AI Features

❌ **Custom LLM Training**

- Not building custom transformer models
- Using pre-built LLMs via APIs

❌ **Advanced Reasoning Systems**

- Chain-of-thought reasoning (can be added later)
- Multi-step problem solving (can be added later)

❌ **Distributed AI Systems**

- Multi-node LLM deployment
- Model sharding
- Federated learning

2. Advanced Safety Features

❌ **Content Moderation**

- Toxicity detection (can be added later)
- Bias detection (can be added later)
- Jailbreak prevention (can be added later)

❌ **Compliance Features**

- GDPR compliance (basic support only)
- HIPAA compliance
- PCI DSS compliance

3. Advanced Integrations

❌ **Third-Party Integrations**

- Slack integration
- Teams integration
- Discord integration
- Custom webhooks

❌ **Payment Processing**

- Stripe integration
- PayPal integration
- Subscription management

❌ **Email Notifications**

- Email alerts
- Newsletter
- Email verification

4. Advanced Analytics

❌ **Advanced Analytics**

- User behavior analytics
- Conversation analytics
- LLM performance analytics
- Cost analytics

❌ **Business Intelligence**

- Custom dashboards
- Data export
- Report generation

5. Multi-Language Support

❌ **Localization**

- Multiple language support
- RTL language support
- Translation management

6. Voice Features

❌ **Voice Chat**

- Speech-to-text
- Text-to-speech
- Voice messaging

❌ **Video Integration**

- Video chat
- Screen sharing
- Recording

7. Advanced Collaboration

❌ **Team Features**

- Team management
- Shared conversations
- Collaboration tools
- Comments and mentions

❌ **Admin Panel**

- User management
- Conversation moderation
- Analytics dashboard
- System configuration

8. Mobile Apps

❌ **Native Mobile Apps**

- iOS app
- Android app
- Desktop app

(Widget works on mobile web)

9. Advanced Caching

❌ **Distributed Caching**

- Redis cluster
- Cache invalidation strategies
- Cache warming

10. Advanced Deployment

❌ **Kubernetes Orchestration**

- Helm charts

- Service mesh
- Auto-scaling policies

❌ **Advanced DevOps**

- Infrastructure as Code (Terraform)
- GitOps
- Advanced monitoring (Prometheus, Grafana)

Deliverables

Phase 1: Core Backend (Weeks 1-3)

- ✅ FastAPI application setup
- ✅ PostgreSQL database schema
- ✅ Authentication system (JWT)
- ✅ User management endpoints
- ✅ Conversation management endpoints
- ✅ Message endpoints
- ✅ Basic error handling
- ✅ API documentation (Swagger)

Phase 2: LLM & RAG Integration (Weeks 4-5)

- ✅ Google Gemini API integration
- ✅ Document parsing (PDF, TXT, DOCX, XLSX, Markdown)
- ✅ Text chunking
- ✅ Embedding generation
- ✅ FAISS vector store
- ✅ Semantic search
- ✅ RAG pipeline

Phase 3: File Management (Week 6)

- ✅ File upload endpoints
- ✅ File validation

- ✅ File storage (S3 or local)
- ✅ File preview
- ✅ File deletion

Phase 4: Frontend - Widget (Weeks 7-8)

- ✅ Floating sphere widget
- ✅ Compact widget view
- ✅ Expanded full-screen view
- ✅ Chat interface
- ✅ Message editing
- ✅ File upload UI
- ✅ Responsive design

Phase 5: Frontend - Dashboard (Week 9)

- ✅ Conversation list
- ✅ Search and filter
- ✅ Conversation preview
- ✅ Settings panel
- ✅ File management

Phase 6: Testing & QA (Week 10)

- ✅ Unit tests
- ✅ Integration tests
- ✅ End-to-end tests
- ✅ Performance testing
- ✅ Security testing
- ✅ Bug fixes

Phase 7: Deployment & Documentation (Weeks 11-12)

- ✅ Docker setup
- ✅ CI/CD pipeline
- ✅ Production deployment

- ✅ Documentation
- ✅ User guides
- ✅ Training

Timeline & Phases

Development Timeline

Plain Text

```
Week 1-3:   Backend Core
Week 4-5:   LLM & RAG
Week 6:     File Management
Week 7-8:   Frontend Widget
Week 9:     Dashboard
Week 10:    Testing & QA
Week 11-12: Deployment & Docs

Total: 12 weeks (8-12 weeks estimated)
```

Milestones

Milestone	Target Date	Deliverable
Backend MVP	Week 3	API endpoints working
LLM Integration	Week 5	Chat with document analysis
Frontend MVP	Week 9	Widget and dashboard
Beta Release	Week 10	Feature complete, testing
Production Release	Week 12	Fully deployed

Success Criteria

Functional Requirements

- ✅ Widget integrates into any website with single script tag

- ✅ Chat responds within 500ms
- ✅ File upload works for all supported formats
- ✅ Conversation history persists
- ✅ Message editing works correctly
- ✅ File generation produces valid files
- ✅ Dashboard displays all conversations
- ✅ Search and filter work accurately

Performance Requirements

- ✅ Chat response time < 500ms
- ✅ File upload < 5s for 50MB file
- ✅ Dashboard loads < 2s
- ✅ Widget initialization < 1s
- ✅ Support 1000+ concurrent users

Security Requirements

- ✅ All data encrypted in transit (HTTPS)
- ✅ Passwords hashed with bcrypt
- ✅ JWT tokens expire after 30 minutes
- ✅ Rate limiting prevents abuse
- ✅ SQL injection prevention
- ✅ XSS protection
- ✅ CSRF protection

Quality Requirements

- ✅ 80%+ code coverage
- ✅ Zero critical bugs
- ✅ 99.9% uptime
- ✅ All endpoints documented
- ✅ All features tested

User Experience Requirements

- ✅ Intuitive UI
- ✅ Smooth animations
- ✅ Mobile responsive
- ✅ Accessibility (WCAG 2.1 AA)
- ✅ Fast load times

Constraints & Assumptions

Constraints

Technical Constraints:

- Windows 11 i5 processor (local development)
- 50MB max file size per upload
- 10,000 requests/day rate limit (free tier)
- 30-minute JWT token expiration
- 5-minute conversation timeout

Business Constraints:

- Budget: $5,000-$10,000 (estimated)
- Timeline: 8-12 weeks
- Team: 2-3 developers
- Target: Production ready

Infrastructure Constraints:

- Single database instance (initially)
- Single backend server (initially)
- No multi-region deployment (Phase 2)
- No advanced monitoring (Phase 2)

Assumptions

Technical Assumptions:

- Google Gemini API remains available
- PostgreSQL is suitable for data volume
- FAISS is sufficient for vector search

- Redis is available for caching
- AWS/GCP available for deployment

Business Assumptions:

- Users have Google account or email
- Users accept terms of service
- Users have stable internet connection
- Website owners can add script tag

User Assumptions:

- Users are tech-savvy
- Users understand chat interface
- Users accept data privacy policy
- Users have modern browsers

Risk Management

High-Risk Items

Risk	Impact	Probability	Mitigation
LLM API outage	High	Medium	Multi-model fallback, caching
Database performance	High	Medium	Indexing, query optimization
Security breach	Critical	Low	Encryption, rate limiting, audits
File processing errors	Medium	Medium	Error handling, validation

Medium-Risk Items

Risk	Impact	Probability	Mitigation
Frontend performance	Medium	Medium	Optimization, caching

Integration issues	Medium	Low	Testing, documentation
Scaling challenges	Medium	Low	Load testing, optimization
User adoption	Low	Medium	UX improvements, support

Mitigation Strategies

1. **Regular Testing** - Automated tests catch issues early
2. **Monitoring** - Real-time alerts for problems
3. **Backups** - Daily database backups
4. **Documentation** - Clear guides for troubleshooting
5. **Support Plan** - Quick response to issues

Resource Requirements

Team

- **1 Backend Developer** - FastAPI, PostgreSQL, LLM integration
- **1 Frontend Developer** - React, responsive design
- **1 DevOps/QA Engineer** - Testing, deployment, monitoring
- **1 Project Manager** (part-time) - Coordination, documentation

Technology Stack

Backend:

- Python 3.10+
- FastAPI
- PostgreSQL
- Redis
- Google Gemini API

Frontend:

- React 19

- TypeScript
- Tailwind CSS
- Zustand (state management)

Infrastructure:

- Docker
- AWS/GCP/Azure
- GitHub (version control)
- GitHub Actions (CI/CD)

Budget Breakdown

Item	Cost	Notes
Developer Salaries	$6,000	3 developers × 4 weeks
Cloud Infrastructure	$1,500	AWS/GCP hosting
LLM API Costs	$500	Google Gemini usage
Tools & Services	$500	GitHub, monitoring, etc.
Total	**$8,500**	Estimated

Summary

What's Included

✅ Full-stack chatbot widget
✅ Document analysis with RAG
✅ File generation
✅ Conversation dashboard
✅ Multi-LLM support
✅ Security & authentication
✅ Performance optimization
✅ Comprehensive testing
✅ Production deployment

✅ Complete documentation

What's Not Included

❌ Custom LLM training
❌ Advanced AI reasoning
❌ Content moderation
❌ Third-party integrations
❌ Payment processing
❌ Mobile apps
❌ Multi-language support
❌ Voice features

Next Steps

1. **Approve project scope**
2. **Assemble development team**
3. **Set up development environment**
4. **Begin Phase 1: Backend Core**
5. **Weekly progress reviews**
6. **Adjust scope as needed**

This project scope provides a clear definition of what will be built, ensuring alignment between stakeholders and the development team.